

BWSC2027 Team Operations Manual

solar_ws0129-main

2026-06-17

1 この文書の目的

この文書は、和歌山大学ソーラーカーチームのメンバーが、**本パッケージを作者不在でも理解し、準備し、運用し、改善できる状態**になるための引き継ぎ資料である。

対象読者は次のとおりである。

- 戦略担当
- 伴走車オペレータ
- 車両制御・マイコン担当
- 実測・同定担当
- 新規加入メンバー

ゴールは次の5点である。

1. workspace が何をしているかを理解する
2. 何が未確定で、何を集める必要があるかを理解する
3. PowerShell から各モードを起動・停止できる
4. live_wifi を BWSC2027 本番運用へつなげられる
5. dashboard を見て、正常・異常・次に取り組むべき行動を判断できる

2 BWSC2027 の公式前提

このパッケージは BWSC2027 を想定して整理しているが、**大会固有値は必ず公式資料で再確認**すること。

2026 年 6 月 17 日時点で、公式サイトで確認できる重要事項は次である。

- 開催期間: 2027-08-22 から 2027-08-29
- ルート: Darwin->Adelaide
- ルート長の目安: 約 3,020km
- Team Manager's Guide によれば、2027 route notes は **2027 年 6 月から電子配布予定**

公式資料:

- Event Regulations: <https://worldsolarchallenge.org/2027-event/regulations>
- 2027 Event Regulations PDF: <https://assets.worldsolarchallenge.org/app/uploads/2026/05/06130905/2027-BWSC-Event-Regulations-V1.0-Published-07052026.pdf>
- 2027 Team Manager's Guide PDF: <https://assets.worldsolarchallenge.org/app/uploads/2026/05/06130911/2027-BWSC-Team-Managers-Guide-V1-Published-07052026.pdf>
- Route Map: <https://worldsolarchallenge.org/about-us/route-map>
- Dates article: <https://worldsolarchallenge.org/latest-news/2027-dates-regulations-announced>

重要な注意:

- 2027 route notes はまだ repo に入っていない
- control stop の最終定義も未反映である
- この repo の route / stop / weather の一部は sample である

3 このパッケージが担う役割

本パッケージは、ソーラーカー戦略運用を次の3層で支える。

3.1 事前準備

- 実測データ整理
- 同定
- route/profile/map の生成
- 天候予報 CSV 取得
- 大会全体シミュレーション

3.2 本番ライブ運用

- 車両状態受信
- 伴走車状態受信
- 天候 API 自動取得
- 風予報のオンライン補正
- 上位プラン再計画
- 下位速度司令 1 Hz 生成
- ソーラーカー / 伴走車への指令送信
- dashboard 表示
- ログ保存

3.3 事後改善

- ログ再解析
- map / model 更新
- 次回シミュレーション精度向上

4 主要モード

通常入口:

- 実行: `.\SolarSim.ps1`
- 設定: `config/solar/bwsc_2027_demo.yaml`

Mode	目的	主 launch
sim	画面付きシミュレーション確認	<code>launch/solarcar_sim.launch.py</code>
measure	実測ログ・地図取得	<code>launch / solar _ measurement . launch.py</code>
live	ROS topic 直結の本番運用	<code>launch / solar _ race _ live . launch.py</code>
live_wifi	WiFi/UDP 文字列連携付き本番運用	<code>launch/solar_race_live_wifi . launch.py</code>

最重要コマンド:

```
.\SolarSim.ps1 -Action build
.\SolarSim.ps1 -Action simulate
.\SolarSim.ps1 -Action forecast
.\SolarSim.ps1 -Action identify
.\SolarSim.ps1 -Mode measure -Action up
.\SolarSim.ps1 -Mode live_wifi -Action up
.\SolarSim.ps1 -Mode live_wifi -Action graph
.\SolarSim.ps1 -Mode live_wifi -Action stop
```

5 ディレクトリの見方

パス	役割
<code>config/solar/</code>	運用設定
<code>launch/</code>	ROS2 launch
<code>mpc_solarcar/</code>	ROS2 node 本体
<code>dashboard/</code>	dashboard フロント
<code>data/route/</code>	route waypoint / route profile / speed profile
<code>data/weather/</code>	offline 予報 CSV
<code>data/race/</code>	control stop / drive schedule

パス	役割
maps/ templates identification/ templates/network/ scripts/ outputs/ docs/	効率 map / Rint / panel map / 同定 CSV 雛形 JSON / systemd / ESP32 雛形 simulate / identify / forecast / sender example / graph export 生成物とログ この資料群

6 主な ROS ノードと役割

Node	役割	主入力	主出力
mpc_node	上位・下位プランナ本体	forecast, route, battery, vehicle state	/planner / speed _ cmd, /planner/summary
telemetry _ text _ bridge_node	UDP JSON と ROS topic の橋渡し	UDP packet, planner topics	vehicle/chase topics, planner UDP
weather_fetch_node	Open-Meteo 予報取得	chase GPS or fallback lat/lon	raw forecast CSV
wind _ correction _ node	風予報のオンライン補正	raw forecast, route, vehicle/chase wind	corrected forecast CSV, /planner / wind_state
solar_autocal_node	走行中の自動校正	measured power, G POA, speed	calibration state
distance_node	s_km が無いときの距離積分	/vehicle / speed _ kmh	/vehicle/s_km
grade_node	slope 推定	GPS, altitude	slope 関連 topic
speed _ command _ bridge_node	planner command を車両側へ再配信	planner speed / mode	vehicle command
dashboard_node	HTTP dashboard	planner / telemetry topics	http : / / localhost:8080
logger_node	CSV logging	各種 topic	outputs / logs / *. csv

7 主な offline script

Script	目的
scripts/solar_sim.py	大会全体シミュレーション
scripts / fetch _ weather _ forecast.py	forecast CSV 取得
scripts / run _ identification _ pipeline.py	同定一括実行

Script	目的
scripts/build_route_profile_from_gps.py	GPS から route 再構成
scripts/build_ocv_curve.py	OCV-SOC カーブ生成
scripts/build_rint_map_from_timeseries.py	Rint map 生成
scripts/build_pv_maps_from_csv.py	panel / MPPT map 生成
scripts/fit_vehicle_params.py	CdA, Crr, P_aux などの再推定
scripts/wifi_vehicle_sender_example.py	車両送信例
scripts / wifi_chase_sender_example.py	伴走車送信例
scripts / wifi_planner_receiver_example.py	planner packet 受信確認

8 現在の repo でできること / 未確定なこと

8.1 すでにできること

- PowerShell から build / simulate / measure / live / live_wifi / graph を実行できる
- live_wifi で UDP JSON 受信・送信ができる
- dashboard が 1 画面で見やすく表示される
- 風補正ロジックが live forecast に組み込まれている
- sample sender で dry run ができる
- PDF 資料が生成できる

8.2 まだ本番投入前に埋める必要があること

項目	現在の状態	最終的に必要なもの
route_waypoints_csv	sample	2027 実ルート + 実測補正版
route_profile_csv	sample	実測から生成した slope/profile
speed_profile_csv	sample	route notes / 制限速度反映版
forecast_csv	sample + Open-Meteo	本番方針を決めた forecast workflow
stop_yaml	sample	2027 route notes 反映版
drive_schedule_yaml	2027 仮版	2027 公式条件とチーム運用方針に整合した版
model.*	初期値	実車同定済み値
maps/*.csv	既存ファイルあり	由来確認済み・最新版
ocv_soc_map	空欄	必要なら生成、不要なら方針明記
WiFi IP/port	例示値	本番ネットワーク設計確定版
sender program	example のみ	実センサ読み出し込みの本番版

項目	現在の状態	最終的に必要なもの
watchdog 方針	文書化のみ	実装・検証済み

9 BWSC2027 までに集めるべき情報

9.1 最重要の不足情報

2026 年 6 月 17 日時点で、本番運用に対して最も足りていないものは次である。

1. 2027 official route notes
2. 最終 control stop 情報
3. 実車の最新同定結果
4. 本番 sender / receiver の実装
5. 現地ネットワーク設計
6. 走行当日の運用ルール

9.2 収集チェックリスト

分類	集めるもの	ソース	反映先
公式	2027 Regulations 最終版	BWSC official	docs/, 運用手順
公式	2027 Team Manager's Guide 最終版	BWSC official	docs/, 物流手順
公式	2027 Route Notes	BWSC official	data / route/, data / race/
ルート	waypoint, altitude, road note	実測 + route notes	route _ waypoints _ csv, route_profile _ csv
速度制限	区間速度制限	route notes	speed_profile_csv
停止情報	control stop open/close, dwell	route notes	stop_yaml
車両	mass, CdA, Crr	coastdown / fit	config / solar / * . yaml
駆動	drive / regen map	実機測定	maps/drive_*, maps/ regen_*
電池	OCV, Rint, temperature effect	bench / log	maps/, ocv_soc_map
太陽	panel eff, MPPT eff	sweep / log	maps/panel_*, maps/ mppt_*
天候	forecast provider 方針	チーム運用判断	live.weather.*
風	wind tuning parameter	dry run / log	live.wind_model.*
通信	planner PC / solar Pi / chase Pi IP	チーム設計	live.wifi_bridge.*
通信	JSON field mapping	firmware / Pi 実装	sender / receiver program
安全	comm lost 時の速度方針	チーム運用判断	firmware と手順書

10 実測・同定で最低限必要なデータ

10.1 走行ログ

最低限:

- time
- speed_kmh
- batt_voltage_v
- batt_current_a
- soc
- batt_temp_c
- lat
- lon
- alt_m

強く推奨:

- s_km
- G_poa
- Tamb_C
- Tcell_C
- wind_speed_ms
- wind_dir_deg
- course_deg
- headwind_ms

10.2 ベンチ / 実験データ

必要度が高い順:

1. battery rest data
2. battery pulse data
3. panel / MPPT sweep data
4. long drive timeseries
5. GPS route survey

10.3 取得時の注意

- すべて UTC または `timezone` を明記する
- 単位を列名に埋め込む
- 欠損は `NaN` または空欄で統一する
- `sensor` 名を `mode` ごとに変えない
- 1 ファイル 1 セッション原則にする

11 推奨するチーム内役割分担

役割	主責務
Strategy Lead	<code>config</code> , <code>forecast</code> , <code>route</code> , <code>schedule</code> , 最終 <code>run</code> 判断
Telemetry Lead	Pi 通信、 <code>dashboard</code> 、 <code>network</code> 状態監視
Vehicle Control Lead	ESP32/マイコンで <code>command</code> 受信、 <code>watchdog</code> 、 <code>clamp</code>
Data/Identification Lead	<code>logs</code> 整理、同定、 <code>map</code> 更新
Chase Operator	伴走車画面監視、通報、ログ保全
Driver Liaison	<code>driver</code> への速度指示伝達、異常時判断支援

12 セットアップ手順

12.1 PC 側

必要:

- Windows
- WSL2
- Ubuntu 22.04
- ROS2 Humble
- `colcon`
- `python3-numpy`, `python3-scipy`, `python3-pandas`, `python3-yaml`, `python3-matplotlib`, `python3-can`, `python3-casadi`

依存関係の定義元:

- `package.xml`
- `setup.py`

12.2 初回 build

```
.\SolarSim.ps1 -Action build
```

期待結果:

- install/
- build/
- log/

が更新される。

13 モード別の動かし方

13.1 大会全体シミュレーション

```
.\SolarSim.ps1 -Action forecast  
.\SolarSim.ps1 -Action simulate
```

見るもの:

- outputs/prerace/*_upper_plan.csv
- 到着時刻
- stop 到達 SoC
- 終了時 SoC
- 平均速度

13.2 実測モード

```
.\SolarSim.ps1 -Mode measure -Action up  
.\SolarSim.ps1 -Mode measure -Action stop
```

目的:

- 生ログ取得
- route/profile 用 GPS 取得
- slope 推定

13.3 本番ライブ WiFi モード

```
.\SolarSim.ps1 -Mode live_wifi -Action up  
.\SolarSim.ps1 -Mode live_wifi -Action stop  
.\SolarSim.ps1 -Mode live_wifi -Action graph
```

13.4 受け入れ試験

live_wifi 起動後、最低限次を満たせば運用開始可と判断しやすい。

1. dashboard が開く
2. networkstatus が更新される
3. vehicleGPS と chaseGPS が埋まる
4. speedcommand と commandstatus が出る
5. forecaststatus が更新される
6. windmodelstatus が source= 付きで出る
7. outputs/runtime/live_forecast_corrected.csv が更新される
8. outputs/logs/solar_live_*.csv が生成される

14 live_wifi の通信方式

transport:

- UDP
- UTF-8
- 1 datagram = 1 JSON

推奨 IP:

- Planner PC: 192.168.50.10
- Solar Pi: 192.168.50.21
- Chase Pi: 192.168.50.22

推奨 port:

- 52001: vehicle/chase → planner
- 52002: planner → solar
- 52003: planner → chase

送信例ファイル:

- templates/network/vehicle_state_example.json
- templates/network/chase_state_example.json
- templates/network/planner_command_example.json
- templates/network/esp32_planner_receiver_example.ino

15 走行前日のチェックリスト

1. 公式 route notes の反映を確認する
2. stop_yaml と speed_profile_csv を更新済みか確認する
3. model.* と maps/*.csv が最新か確認する
4. Pi / ESP32 側の IP と port が一致しているか確認する
5. live_wifi dry run を実施する
6. graph を出して topic 接続を確認する
7. ログ保存先の空き容量を確認する
8. fallback GPS のまま forecast を取っていないか確認する

16 レース当日の運用フロー

16.1 朝

1. PC 起動
2. Pi/マイコン起動
3. WiFi 接続確認
4. .\SolarSim.ps1-Modelive_wifi-Actionup
5. dashboard で networkstatus と forecaststatus を確認
6. planner command の受信確認

16.2 走行中

見る優先度:

1. SpeedCommand と meas
2. SoC
3. WindPlan
4. NextStop
5. NetworkStatus
6. CalibrationStatus

16.3 昼・風向変化時

- windobs
- windforecast
- windplan
- windmodelstatus

を重点的に見る。

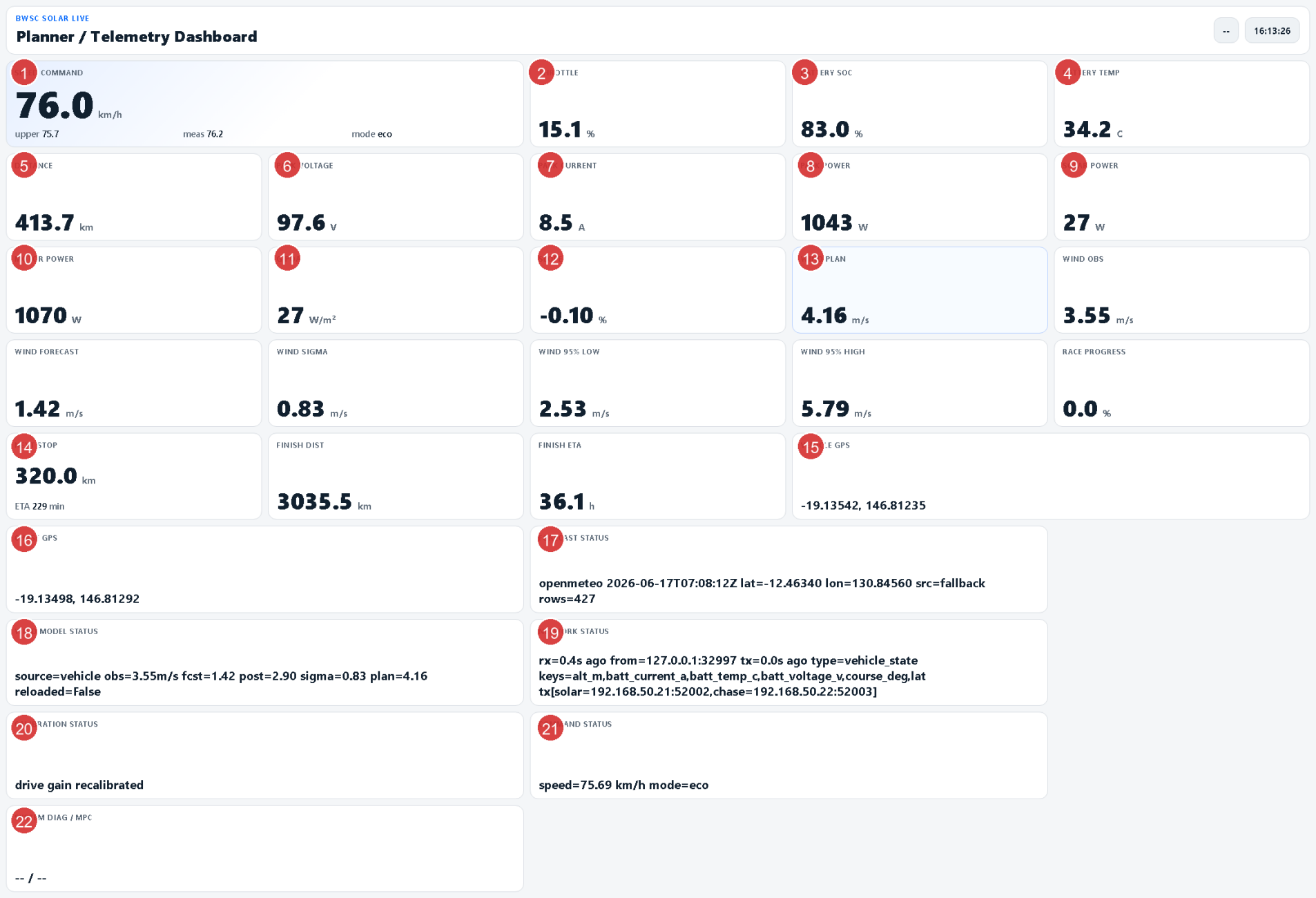
16.4 日没・終了時

1. log を閉じる
2. `outputs/logs/*.csv` を退避
3. `forecast` と `corrected forecast` を保存
4. 異常があれば時刻と状況をメモ

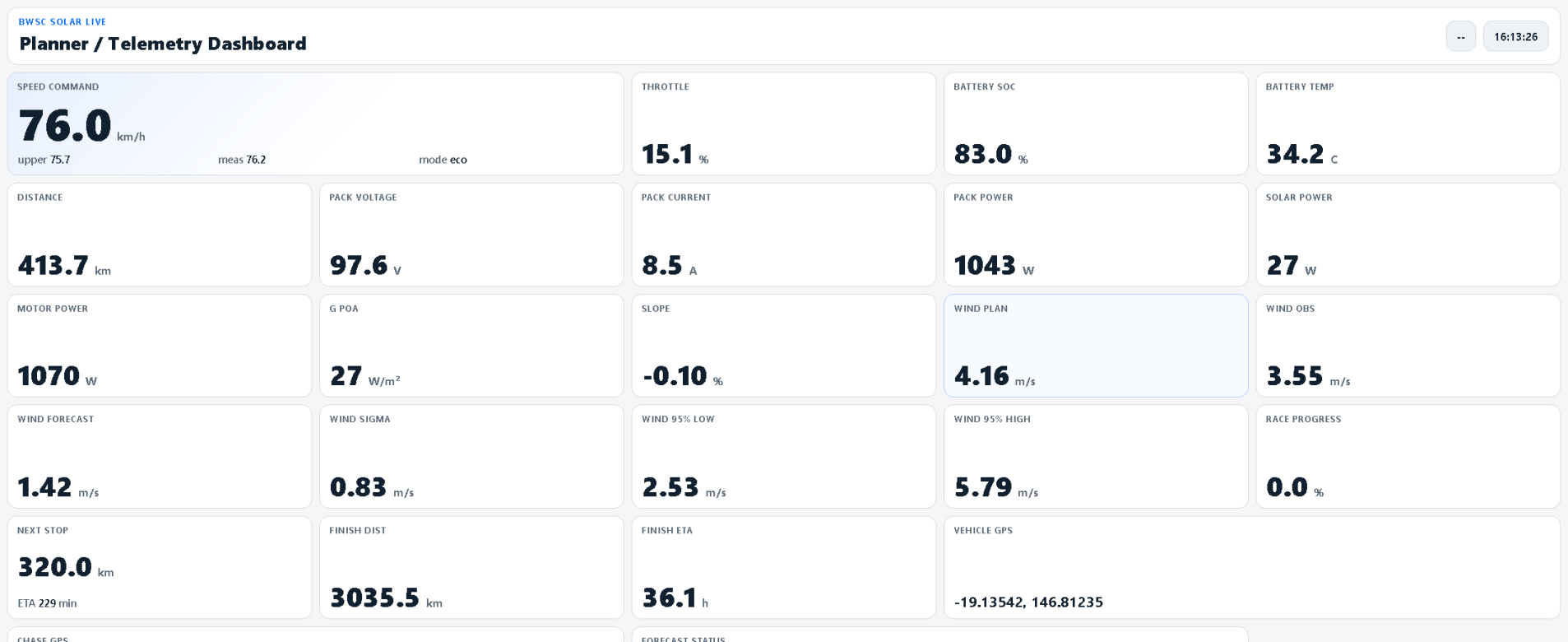
17 dashboard 完全解説

この章のスクリーンショットは、**2026-06-17 の dry run** で `live_wifi` を起動し、`sample sender` で値を流した例である。本番の数値は当然変わる。重要なのは **どの値をどう読むか** である。

dashboard 全体図



dashboard 上半分拡大



dashboard 下半分拡大

CHASE GPS

-19.13498, 146.81292

WIND MODEL STATUS

source=vehicle obs=3.55m/s fcst=1.42 post=2.90 sigma=0.83 plan=4.16
reloaded=False

CALIBRATION STATUS

drive gain recalibrated

SYSTEM DIAG / MPC

-- / --

FORECAST STATUS

openmeteo 2026-06-17T07:08:12Z lat=-12.46340 lon=130.84560 src=fallback
rows=427

NETWORK STATUS

rx=0.4s ago from=127.0.0.1:32997 tx=0.0s ago type=vehicle_state
keys=alt_m,batt_current_a,batt_temp_c,batt_voltage_v,course_deg,lat
tx[solar=192.168.50.21:52002,chase=192.168.50.22:52003]

COMMAND STATUS

speed=75.69 km/h mode=eco

18 番号ごとの説明

No.	表示	何を表すか	正常時の見方	異常/注意時の行動
1	Speed Command	下位 planner が車両へ出した速度司令	upper, meas, mode と合わせて見る	meas が追従しないなら車両側追従不良または clamp を疑う
2	Throttle	planner が計算した下位入力	高いほど駆動要求が大きい	100% 張り付きなら上位要求過大か車両追従不足
3	Battery SoC	バッテリー残量	目標 SoC に対して十分かを見る	急減時は speed command を下げる判断材料
4	Battery Temp	電池温度	運用温度帯か確認	上昇しすぎなら保護・制限を検討
5	Distance	現在地点の route 距離 s_km	next stop / finish と整合しているか	不自然なら GPS / distance node を確認
6	Pack Voltage	パック電圧	SoC や負荷と整合しているか	急低下なら負荷過大や電池異常を疑う
7	Pack Current	パック電流	消費傾向の把握	高止まりなら過大消費を疑う
8	Pack Power	パック電力	実質的な電池出力	想定以上なら speed / slope / headwind を確認
9	Solar Power	太陽から入っている電力	日射と整合するか	低すぎるなら雲・角度・センサ・パネル異常候補
10	Motor Power	駆動電力	Pack Power と比較して損失感を掴む	急上昇なら速度指令や勾配を確認
11	G POA	パネル面入射日射	solar power の説明変数	src = fallback 中は forecast 依存であることに注意
12	Slope	現在勾配	負荷増減の説明に使う	不自然なら GPS / altitude / route profile を確認
13	Wind Block	WindPlan, Obs, Forecast, Sigma, 95% 区間のまとめ	planner が何を想定し、どれくらい不確かかを見る	Obs と Forecast がズレ続けるなら tuning 見直し
14	Next Stop / Finish	次 stop までの距離と ETA、finish 残距離/ETA	戦略判断の中心	ETA が崩れたら上位プラン再確認
15	Vehicle GPS	ソーラーカー位置	route 上の現在地確認	空欄なら車両送信が来ていない
16	Chase GPS	伴走車位置	伴走車の計測基準確認	空欄なら chase sender を確認
17	Forecast Status	weather node の取得状態	src=gps が理想、rows も確認	src = fallback のまま本番運用しない
18	Wind Model Status	風補正の現在状態	source = vehicle / chase, obs, fcst, post, sigma, plan を読む	waitingobservation なら風入力が出ていない

No.	表示	何を表すか	正常時の見方	異常/注意時の行動
19	Network Status	通信の鮮度と送信元	<code>rx=0.xsago</code> が理想	<code>rx=never</code> や数秒更新なしなら通信断
20	Calibration Status	自動校正の進行状態	<code>collecting</code> は起動直後に自然	異常な補正が続くなら <code>log</code> を見直す
21	Command Status	再配信された車両 <code>command</code> の要約	<code>speed</code> と <code>mode</code> が出ていれば正常	出ないなら <code>command bridge</code> を確認
22	System Diag / MPC	予備の診断欄	将来の診断表示枠	値が入るよう拡張可能

18.1 dashboard で最優先で見べき 6 項目

1. SpeedCommand
2. BatterySoC
3. WindPlan
4. NextStop
5. NetworkStatus
6. ForecastStatus

18.2 -- が出る意味

-- は主に次を意味する。

- `topic` がまだ来ていない
- `sender` 側がその `field` を送っていない
- `live mode` では未使用
- `node` が未計算

本番運用で -- のままにしてよいものと、だめなもの:

項目	-- 許容か
Speed Command	不可
Battery SoC	不可
Vehicle GPS	不可
Network Status	不可
Forecast Status	不可
Wind Model Status	通常は不可
System Diag / MPC	可

19 典型的な異常と対処

症状	まず疑うこと	すぐやること
networkstatus rx=never	が sender 不起動、port 違い、WiFi 断	Pi 側 sender と IP/port を確認
forecaststatus fallback のまま	が src = chase GPS 未受信	chase sender を確認
windmodelwaitingobservation	wind field 未送信	wind_speed_ms, wind_dir_deg, course_deg を確認
meas が speedcommand に追従しない	車両制御側問題、safety clamp	車両 firmware と actual speed を確認
SoC が急減	消費過大、風、勾配、電池異常	speed command を見直し、電池電圧/電流確認
distance が止まる	GPS / speed topic 欠損	/vehicle / speed_kmh と /vehicle/s_kmh 確認
command は出るが車両が動かない	receiver 実装不足	マイコン受信ログと clamp 条件確認

20 チーム引き継ぎの受け入れ基準

この資料を使った引き継ぎが完了したと判断する条件:

1. 新メンバーが `simulate` を単独実行できる
2. 新メンバーが `measureup` を単独実行できる
3. 新メンバーが `live_wifiup` を単独実行できる
4. 新メンバーが dashboard の主要 10 項目を説明できる
5. 新メンバーが sender JSON の必須 field を説明できる
6. 新メンバーが `route`, `weather`, `stop`, `maps` の差し替え場所を知っている
7. 新メンバーが `sample` のままなものを列挙できる

21 運用開始前に必ず更新すべきファイル

- `config/solar/bwsc_2027_demo.yaml`
- `data/route/bwsc_route_waypoints_*.csv`
- `data/route/bwsc_route_profile_*.csv`
- `data/route/bwsc_speed_profile_*.csv`
- `data/race/bwsc_control_stops_*.yaml`
- `data/weather/*.csv` または live forecast 方針
- `maps/*.csv`
- Pi / ESP32 の sender / receiver 実装

22 この資料と併用する文書

- docs/solar_workflow.pdf
- docs/solar_live_wifi_manual.pdf
- rqt_graph_solar_live_wifi.png
- BWSC official regulations
- BWSC Team Manager's Guide
- 2027 route notes

23 最終メッセージ

このパッケージは、**そのまま魔法のように勝手に正解を出すものではない**。強いのは次の3点である。

1. 情報を1か所に集めて見える化できること
2. シミュレーションと本番を同じ設計思想で回せること
3. 実測を次回改善へ戻せること

逆に、最終的な勝負を決めるのは次である。

- route / stop / weather の入力品質
- vehicle model と map の品質
- 通信実装の安定性
- チームが dashboard を読んで判断できること

この文書を使って、**全員が「自分でも動かせる」と言える状態**まで持って行ってほしい。